HIGH PERFORMANCE COMPUTING SYSTEMS

Architectures, Parallelism, and the Scale of Modern AI

Comprehensive Study Guide — Lecture 1

Lecturer: Abdelhakim Cheriet
National School of Artificial Intelligence
4th Year — Data Science & AI Engineering
Academic Year: 2025–2026

Introduction

This study guide is based on Lecture 1 of the High Performance Computing (HPC) Systems course. It covers the landscape of the world's most powerful computers, the fundamental principles behind parallel computing, the programming models used in HPC, the scientific applications that motivate this field, and the key algorithmic patterns (motifs) that drive hardware and software co-design. By the end of this guide, you should understand why parallelism is no longer optional in modern computing, and how the hardware, software, and application layers interact to deliver extreme performance.

1. Measuring Supercomputer Performance: The TOP500 List

1.1 What Is the TOP500?

The TOP500 project is an international effort that ranks the 500 most powerful non-distributed computer systems in the world. The list is published twice a year, in June and November, providing a reliable snapshot of the evolving state of high-performance computing (HPC). It is the go-to reference for researchers, engineers, and governments who want to benchmark or compare national and institutional computing capabilities.

The Linpack Benchmark (HPL)

Each system on the TOP500 is ranked by its performance on the Linpack benchmark, formally known as the High Performance Linpack (HPL). The benchmark measures how fast a computer can solve a dense system of linear equations of the form Ax = b, where A is a large matrix. The result is expressed in FLOP/s (Floating Point Operations per Second).

> **Key Insight**
> A single 'Flop' is one basic floating-point operation (e.g., addition or multiplication). 'Rmax' is the maximum LINPACK performance actually achieved by the machine. Performance tiers go: Giga (10^9) → Tera (10^12) → Peta (10^15) → Exa (10^18). As of 2024, the fastest machines have entered the Exascale era (> 10^18 FLOP/s).

1.2 The Current Top 4 Supercomputers (November 2024)

The following table summarises the top-ranked machines as covered in the lecture. These systems illustrate the diversity of architectural approaches to achieving extreme performance.

Rank	System / Key Facts

#1 El Capitan	HPE Cray EX255a at Lawrence Livermore National Lab. Rmax: 1.809 EFlop/s. Uses AMD Instinct MI300A APU — a novel design that fuses CPU (Zen 4) and GPU (CDNA 3) on a single package with unified HBM3 memory access (256 GB per node). This eliminates the PCIe bottleneck. Total cores: ~11.34 million. Power: 29.7 MW. Efficiency: 60.9 GFlops/Watt.
#2 Frontier	HPE Cray EX235a at Oak Ridge National Lab. Rmax: 1.353 EFlop/s. Discrete architecture: 1 AMD EPYC CPU driving 4 AMD Instinct MI250X GPUs per node. Connected via Slingshot-11 interconnect. Total cores: ~9.07 million. Power: 24.6 MW. Launched 2022 — the world's first Exascale machine.
#3 Aurora	Intel Exascale Compute Blade at Argonne National Lab. Rmax: 1.012 EFlop/s. Uses Intel Xeon CPU Max + Intel Data Center GPU Max (Ponte Vecchio). Notable for its MxP (Mixed Precision) benchmark score of 11.6 EFlop/s — exceptional for AI workloads. Power draw: 38.7 MW (highest in the top 5). Cores: ~9.26 million.
#4 JUPITER Booster	Eviden BullSequana XH3000 at FZJ, Germany. Rmax: 1.000 EFlop/s. Uses NVIDIA Grace CPU + Hopper GPU connected via NVLink-C2C at 900 GB/s. Power: only 15.8 MW — making it the most energy-efficient Exascale system (roughly half the power of El Capitan). Cores: ~4.8 million.

Important Concept: The APU Advantage (El Capitan)

Traditional GPU-accelerated systems place the CPU and GPU on separate chips, communicating over a PCIe bus (bandwidth ~64 GB/s). El Capitan uses an APU (Accelerated Processing Unit), which fuses both on one die sharing HBM3 memory. This removes the PCIe bottleneck and enables seamless data sharing, critical for multi-physics simulations where memory access patterns are irregular.

2. What Is a Parallel Computer?

2.1 The Two Motivations: Speed and Memory

Before defining what a parallel computer is, it helps to understand why we need one. There are two fundamental drivers:

- The Need for Speed: Some scientific problems require computing the prime factors of a billion numbers, simulating climate models, or training large neural networks. A single processor running sequentially would take years. With one million processors working in parallel on independent sub-tasks, the same work can be done in a fraction of the time.
- The Need for Memory: Some problems simply cannot fit in the RAM of a single machine. If a simulation requires more than 100 TB of memory (e.g., a full-brain neural simulation or a global climate model), the data must be distributed across hundreds or thousands of nodes.

> **Key Insight**
> Bill Dally (Chief Scientist at NVIDIA and Professor at Stanford) summarised this elegantly: 'Performance = Parallelism; Efficiency = Locality.' This means: to go fast, you need to do many things at once; to be efficient, you need to keep data physically close to the processor that uses it.

2.2 Parallel Reduction: A Classic Example

To illustrate the power of parallelism, consider summing n numbers. Serially, this takes O(n) steps. In parallel, we can exploit the associativity of addition to reduce the problem in O(log n) steps using a tree-based approach:

Given 8 values [1, 3, 1, 0, 4, -6, 3, 2], in the first step 4 processors each add a pair. In the second step, 2 processors each add a pair of previous results. In the third step, 1 processor adds the final two values to get the total (8). This reduces 8 operations into 3 parallel steps ($\log_2 8 = 3$). The trade-off is that it uses n processors simultaneously.

2.3 The AI Memory Wall

A critical illustration of the memory problem is the 'AI Memory Wall.' Transformer model sizes have grown at approximately 410× every two years, while GPU memory has only doubled every two years. As shown in the lecture chart (Transformer Model Size vs. AI Hardware Memory), by 2020, models like GPT-3 (175 billion parameters) already exceed the memory of a single GPU by orders of magnitude. This gap makes distributed training across multiple GPUs — and entire clusters — mandatory for modern large language models.

3. Classifying Parallel Hardware: Flynn's Taxonomy

Flynn's Taxonomy (1966) is a widely-used classification scheme for computer architectures, based on the number of simultaneous instruction streams and data streams.

Category	Description & Examples
SISD (Single Instruction, Single Data)	The classical serial computer. One instruction operates on one data point at a time. Example: a traditional single-core CPU executing a loop sequentially. This is what all code ran on before multi-core processors.
SIMD (Single Instruction, Multiple Data)	One instruction is applied simultaneously to a vector of data items. This is the GPU model and also how CPU vector extensions (SSE, AVX) work. A single ADD instruction can add 8, 16, or even 32 pairs of numbers at once. Critical for AI and matrix multiplication (the backbone of deep learning).

MIMD (Multiple Instruction, Multiple Data)	Independent processors executing entirely different code paths on different data simultaneously. This is the cluster model: each node in an HPC cluster can be running a different part of the algorithm. Most real-world supercomputers are MIMD at the cluster level.

> **Key Insight**
>
> Modern HPC systems are hybrids. At the node level, the GPU provides SIMD-style massive parallelism. Across nodes, the cluster operates in MIMD fashion. This hierarchy is what gives modern supercomputers their extreme capability.

4. Memory Architecture: Shared vs. Distributed

4.1 Shared Memory (SMP — Symmetric Multiprocessing)

In a shared memory system, multiple processors all have access to a single, unified global memory pool. They communicate by reading and writing to the same memory addresses. Think of it like multiple workers sharing a single whiteboard: anyone can read or write anything.

- Advantage: Easy to program. Threads can share data without explicit messaging.
- Disadvantage: Limited scalability. As more processors are added, they all contend for the shared memory bus, causing bottlenecks (bus contention). Practical limit is typically a few hundred cores on a single node.

4.2 Distributed Memory (Clusters)

In a distributed memory system, each processor (or node) has its own private local memory. Nodes are connected by a high-speed network. To share data, a node must explicitly send a message over the network. Think of it like workers in different offices: each has their own desk, but they must call each other to share information.

- Advantage: Virtually infinite scalability — you can keep adding nodes.
- Disadvantage: Programmer must explicitly manage all data movement and synchronisation. This is significantly more complex to program correctly.

> **Important Concept: The Hybrid Reality**
>
> Modern HPC clusters are distributed systems composed of shared memory nodes. Each node (e.g., a dual-socket server with 64 cores) is a shared memory machine internally. Nodes communicate across the cluster via a high-speed network (e.g., InfiniBand at 100+ Gb/s). This is why HPC programmers often combine OpenMP (for within-node parallelism) with MPI (for across-node communication).

5. Programming Models: The Software Layer

Given the hardware landscape described above, three dominant programming models have emerged to exploit parallelism at different levels of the system.

5.1 Tool 1 — OpenMP (Open Multi-Processing)

OpenMP is designed for shared memory parallelism within a single node. It uses a directive-based approach: the programmer adds special compiler hints (pragmas) to existing C, C++, or Fortran code, and the compiler generates multi-threaded code automatically.

- Target: Shared memory systems (single node, multiple cores).
- Mechanism: Compiler directives (#pragma omp ...).
- Concept: Multithreading within a single process — all threads share the same memory space.
- Primary Use Case: Parallelising 'for' loops and accelerating local matrix operations.

The Fork-Join Execution Model

OpenMP follows the fork-join model. The program begins as a single master thread. When it reaches a parallel region (marked by a pragma), it 'forks' a team of worker threads. All threads execute the region concurrently. At the end of the region, all threads synchronise at an implicit barrier and 'join' back into a single master thread that continues execution.

Demo 1: Hello World in OpenMP

The simplest OpenMP program demonstrates thread creation:

```
#include <stdio.h>
#include <omp.h>

int main() {
    // #pragma omp parallel spawns a team of threads
    #pragma omp parallel
    {
        int id    = omp get thread num();   // unique thread ID
        int total = omp_get_num_threads();  // total threads in team
        printf("Hello from thread %d of %d\n", id, total);
    }
    return 0;
}

// Compile: gcc -fopenmp hello.c -o hello
// Run:     export OMP_NUM_THREADS=4 && ./hello

// Output (order is non-deterministic):
```

```
// Hello from thread 1 of 4
// Hello from thread 0 of 4
// Hello from thread 3 of 4
// Hello from thread 2 of 4
```

The key observation is that the output order is non-deterministic — the OS schedules threads independently. This is a fundamental characteristic of parallel programs: you cannot assume execution order unless you explicitly synchronise.

Demo 2: Parallel For Loop (Work Sharing)

The most common OpenMP pattern is parallelising a for loop. Without OpenMP, each iteration runs serially. With one pragma, the runtime automatically partitions the iterations across threads:

```
// Serial version
for (int i = 0; i < N; i++) {
    a[i] = b[i] + c[i];   // N iterations, done one-by-one
}

// Parallel version — add just ONE line
#pragma omp parallel for
for (int i = 0; i < N; i++) {
    a[i] = b[i] + c[i];   // iterations split across threads automatically
}
// With 4 threads and N=1000: Thread 0 does i=0..249,
// Thread 1 does i=250..499, etc. No manual calculation needed.
```

The Memory Model: Shared vs. Private Variables

OpenMP has two rules for variable scope inside a parallel region:

- Shared (default): Variables declared outside the parallel block exist in the global heap and are accessible by all threads. The risk is data races — two threads writing to the same address simultaneously produces undefined behaviour.
- Private: Variables declared inside the parallel block are allocated on each thread's private stack. Each thread has its own copy, so there is no interference.

> **Key Insight**
>
> A data race is one of the most dangerous bugs in parallel programming. It occurs when two threads access the same memory location simultaneously, and at least one access is a write. The result is non-deterministic and may silently produce wrong answers. OpenMP provides mechanisms like 'reduction', 'critical', and 'atomic' to safely handle shared writes.

5.2 Tool 2 — MPI (Message Passing Interface)

MPI is designed for distributed memory systems — communication across thousands of nodes. Unlike OpenMP, MPI is not directive-based; it is a library of function calls that the programmer explicitly invokes to send and receive data between independent processes.

- Target: Distributed memory systems (clusters of nodes).
- Mechanism: Communication library (MPI_Send, MPI_Recv, collective operations).
- Concept: Multiple independent processes, each with private memory, communicating over a network.
- Scale: Across thousands of nodes — the only way to reach Exascale.
- Challenge: 'Shared Nothing' architecture. The programmer must explicitly manage every byte of data movement and every synchronisation point.

The SPMD Programming Model

MPI programs follow the SPMD (Single Program, Multiple Data) pattern. Every process runs the exact same executable, but each process is assigned a unique integer identifier called its Rank. The process logic branches based on this rank:

```
// Every node runs this same code
if (Rank == 0) {
    distribute data();   // Rank 0 is the 'master': reads input, sends to
workers
} else {
    compute worker();    // All other ranks receive and process their portion
}

// Explicit communication:
MPI_Send(data, count, MPI_DOUBLE, dest, tag, MPI_COMM_WORLD);
MPI_Recv(data, count, MPI_DOUBLE, src,  tag, MPI_COMM_WORLD, &status);
```

The critical point is that data never moves between processes automatically. If Process 0 needs to send the value x to Process 1, it must call MPI_Send, and Process 1 must call MPI_Recv at the same time. This explicit coordination is what makes MPI powerful but also demanding to program correctly.

5.3 Tool 3 — Accelerators & Heterogeneity (CUDA/HIP)

Modern HPC nodes contain not just CPUs, but also GPU accelerators. The CPU (host) handles logic, control flow, and I/O. The GPU (device) handles massive parallel arithmetic — specifically the matrix operations that dominate AI and scientific computing.

- CPU: A few powerful cores (8–128) optimised for serial logic and low-latency tasks.
- GPU: Thousands of simpler cores (up to ~18,000 on an H100) optimised for throughput — executing the same operation on millions of data items simultaneously (SIMT: Single Instruction, Multiple Threads).
- Integration: CUDA (NVIDIA), HIP (AMD), OpenMP Target Offloading.

- Interconnect: Data moves from CPU memory to GPU memory via PCIe (~64 GB/s) or NVLink (~900 GB/s).

Programming Model	Purpose & Scope
Programming Model	Purpose & Scope
OpenMP	Shared memory parallelism on a single node's CPU cores. Uses #pragma directives.
MPI	Distributed memory communication across all nodes in a cluster. The only path to Exascale.
CUDA / HIP	GPU programming for massive data-parallel kernels (matrix math, convolutions).
OpenMP + MPI (Hybrid)	The standard in modern HPC: OpenMP within a node, MPI between nodes.
MPI + CUDA (Full Stack)	The full stack: MPI across nodes, CUDA within each GPU on each node.

6. Concurrency vs. Parallelism, and What Is NOT a Parallel Computer

6.1 Concurrency vs. Parallelism — A Critical Distinction

These two terms are frequently confused, but they mean fundamentally different things:

Concept	Definition & Analogy
Concurrency	Multiple tasks are logically active at the same time, but only one executes at any given instant. Progress is made via rapid context switching (time-slicing). Analogy: one person juggling three balls — only one ball is ever in their hands at once. Example: a single-core CPU running multiple browser tabs.
Parallelism	Multiple tasks are physically executing at the exact same instant, on different hardware units. Requires multiple physical processing units (cores, processors). Analogy: three people each holding one ball simultaneously. Example: a 4-core CPU computing four loop iterations at exactly the same clock cycle.

> **Key Insight**
> Concurrency is a software concept; parallelism is a hardware reality. A concurrent program can run on one core via time-slicing. A parallel program requires multiple cores. All parallel programs are concurrent, but not all concurrent programs are parallel.

6.2 Parallel Computer vs. Distributed System

Another common confusion is between a parallel computer and a distributed system. The distinction lies in the primary design goal:

- A distributed system is inherently designed to serve geographically distributed clients (e.g., a web server farm, a CDN, or a database cluster). Fault tolerance and availability are key goals.
- A parallel computer is designed to maximise performance for a single computation. It may use distributed memory internally (multiple nodes with private memory), but the goal is speed of computation, not geographic distribution or client service.

The boundary has blurred — for example, cloud HPC platforms use distributed infrastructure to run parallel computations — but the conceptual distinction remains important for understanding design tradeoffs.

7. Science Using High Performance Computing

7.1 Simulation: The Third Pillar of Science

Historically, science rested on two pillars: theory (mathematical models) and experiment (physical observation). In the late 20th century, computational simulation emerged as a 'Third Pillar' — a way to study systems that cannot be fully captured by closed-form equations and cannot be safely or affordably experimented upon.

Why Simulation Is Necessary

High-performance simulation is used to study phenomena that are:

- Too big: Cosmological simulations of galaxy formation.
- Too small: Molecular dynamics and protein folding at the atomic scale.
- Too fast: Supersonic combustion, nuclear reactions.
- Too slow: Climate change over centuries, geological processes.
- Too expensive: Full-scale aircraft testing, crash simulations.
- Too dangerous: Nuclear weapons physics, pandemic spreading models.

7.2 The Fourth and Fifth Paradigms of Science

Data analysis (driven by massive experimental datasets from telescopes, genome sequencers, particle detectors, and sensor networks) is now considered the Fourth Paradigm of Science.

High Performance Data Analytics (HPDA) deals with datasets that are too big, too complex, too noisy, or too fast (streaming) for conventional analysis.

Machine Learning is emerging as a proposed Fifth Paradigm — using data-driven models to discover scientific laws, accelerate simulations (surrogate models), and process experimental data at scales that traditional methods cannot handle. This is the direct link between HPC and modern AI.

Paradigm	Description
Paradigm	Description
Theory	Mathematical modelling and analytical solutions (Newton's laws, Maxwell's equations, etc.)
Experiment	Physical observation and measurement in the real world.
Simulation	Numerical computation to model physical systems (climate, fluids, materials, etc.)
Data Analysis (HPDA)	Mining and processing massive datasets from experiments and sensors.
Machine Learning (?)	Emerging fifth paradigm — AI-driven discovery and simulation acceleration.

7.3 Exascale Applications at Berkeley Lab

Lawrence Berkeley National Laboratory (LBNL) provides a concrete illustration of the breadth of Exascale-class applications. These span disciplines including combustion (clean energy), cosmology (dark matter), genomics (precision medicine), astrophysics (supernova modelling), subsurface flow (carbon sequestration), earthquake simulation (seismic risk), urban systems (city planning), and photon science (advanced light sources). Each of these requires the combined memory, compute, and I/O capabilities of a modern Exascale system.

8. The Motifs of Parallel Computing

8.1 The Colella Paradigm: The Seven Dwarfs

In 2004, Phillip Colella of Berkeley Lab made a landmark observation: despite the enormous diversity of scientific simulations, nearly all of them can be reduced to combinations of just seven fundamental numerical methods. He called these the 'Seven Dwarfs.' This insight was revolutionary because it changed how computer architects think: instead of designing future processors to run legacy benchmark programs, they should design processors to optimally execute the methods that future applications will need.

> **Key Insight**
>
> The key design principle: 'If you understand the Motif, you understand the bottleneck. If you understand the bottleneck, you can architect the solution.' This is why GPUs were designed for Dense Linear Algebra, and why Google built TPUs for AI.

8.2 The Original 7 Dwarfs (HPC Focus)

Motif	Description & Bottleneck
Motif	Description & Bottleneck
Dense Linear Algebra (DLA)	Matrix-matrix multiplication (e.g., training a neural network layer). Pattern: every entry of A multiplies every entry of B to form C. Bottleneck: Compute-bound — high arithmetic intensity. We do lots of math on the same data. Can reach peak theoretical performance. GPUs excel here.
Sparse Linear Algebra (SLA)	Matrix-vector products where the matrix is mostly zeros (e.g., finite element methods, graph problems). Bottleneck: Memory-bound — the CPU stalls waiting for non-zero entries located at irregular memory addresses. Performance often less than 10% of peak.
Spectral Methods (FFT)	Transforming data between time domain and frequency domain (e.g., signal processing, weather models). Pattern: every output depends on every input — requires an 'All-to-All' communication across all processors. Bottleneck: Communication-bound.
N-Body (Particle Methods)	Computing pairwise interactions between discrete particles (gravity, molecular forces, electrostatics). Naive complexity $O(N^2)$ — improved by Barnes-Hut tree algorithm to $O(N \log N)$. Bottleneck: Computationally intensive.
Structured Grids	Regular array computations where spatial neighbours are also memory neighbours (e.g., weather models). Pattern: stencil updates on a regular mesh. Bottleneck: High bandwidth demand.
Unstructured Grids	Irregular meshes that conform to complex geometry (e.g., airflow over an aircraft). Pattern: 'pointer chasing' to find neighbours. Bottleneck: Memory latency and hard to parallelise due to data races on shared mesh nodes.
Monte Carlo Methods	Repeated independent random trials to estimate probabilities (e.g., financial risk, quantum chemistry). Each trial is independent — this is 'Embarrassingly Parallel.' No bottleneck. MapReduce is a generalised version of this pattern.

8.3 The Berkeley Expansion: 13 Motifs

In 2006, the Berkeley Par Lab extended the original 7 Dwarfs to cover not just scientific computing but also the commercial world (games, databases, AI). They added 6 new motifs, bringing the total to 13:

- Combinatorial Logic: High-throughput bit-wise manipulation (cryptography, hashing).
- Graph Traversal: BFS/DFS on large irregular graphs (social networks, web search). Memory latency bound due to random jumps through memory.
- Finite State Machines: Pattern matching and parsing. Often 'Embarrassingly Sequential' — the next state depends on the current one, making parallelism difficult.
- Dynamic Programming: Breaking complex problems into overlapping sub-problems (bioinformatics sequence alignment, optimal control).
- Backtrack / Branch-and-Bound: Search with pruning (chess solvers, constraint satisfaction).
- Graphical Models: Probabilistic inference in Bayesian networks and Markov random fields — the backbone of classical machine learning.

8.4 Hardware/Software Co-Design: The End Game

The motif framework has a profound practical consequence: hardware and software are now co-designed. Modern processor architects do not just make faster general-purpose chips; they analyse the dominant computational motifs of future applications and build specialised hardware to execute those motifs efficiently. Examples include:

- GPUs (NVIDIA, AMD): Designed around Dense Linear Algebra (matrix multiplication) — the dominant motif in deep learning.
- TPUs (Google): Tensor Processing Units, specifically designed for the matrix operations in transformer-based AI models.
- HBM Memory: High Bandwidth Memory, designed to feed the massive data appetite of GPU matrix engines.
- NVLink / Slingshot Interconnects: High-bandwidth, low-latency networks designed to support the All-to-All communication patterns of FFT and large-scale distributed training.

> **Important Concept: Hardware/Software Co-Design**
>
> We no longer write software purely for existing chips. Hardware architects study the motifs that future applications will need, then build chips that execute those motifs efficiently. Software teams then co-design algorithms to exploit the resulting hardware. This bidirectional loop is what drives the performance gains in modern HPC and AI systems.

9. Why All Modern Computers Are Parallel (Since 2005)

9.1 Moore's Law and Its Reinterpretation

Gordon Moore, co-founder of Intel, predicted in 1965 that the number of transistors on a semiconductor chip would double roughly every 18 months. This prediction, known as Moore's

Law, held remarkably well for 50 years, driving exponential performance improvements in computing.

The Power Wall: The End of Frequency Scaling

Until the mid-2000s, increasing transistor density translated directly into higher clock speeds (from MHz in the 1980s to ~3 GHz by 2004). But a physical barrier emerged: power consumption scales as the cube of frequency ($P \propto f^3$). A chip running at 6 GHz would consume four to eight times the power of one at 3 GHz, generating heat that cannot be practically dissipated. Clock speeds stalled at approximately 3 GHz.

The industry's response: instead of making one core faster, put more cores on the chip. Since 2005, every new processor generation has added cores rather than dramatically increasing clock speed. This is Moore's Law Reinterpreted: transistor density still doubles, but the performance gains now come from parallelism, not faster clocks.

> **Key Insight**
>
> Herb Sutter's famous 2005 article called this 'The Free Lunch Is Over.' Before 2005, software automatically ran faster on newer hardware without any changes. After 2005, programmers must explicitly write parallel code to exploit the new hardware. This is the single most important reason why HPC and parallel programming matter to every software engineer, not just supercomputer specialists.

9.2 Parallel Hardware Is Everywhere

Since 2005, parallelism is not just a supercomputer feature — it is the default in all computing devices:

- Laptops and desktops: AMD Ryzen (8–16 cores), Intel Core (8–24 cores).
- Servers: AMD EPYC (up to 128 cores per socket).
- GPUs: NVIDIA H100 (~16,896 CUDA cores), AMD MI300A (~228 compute units).
- Specialised AI chips: Groq LPU (Language Processing Unit), Huawei Ascend NPU.
- Smartphones: Apple A-series and Qualcomm Snapdragon chips have 6–12 CPU cores plus dedicated GPU and NPU cores.

The conclusion is unavoidable: parallel hardware is the default. Any software engineer who writes only serial code is leaving the vast majority of the hardware's capability unused.

10. Amdahl's Law: The Theoretical Limit of Parallelism

10.1 The Formula

Amdahl's Law gives the theoretical maximum speedup achievable by parallelising a program, as a function of the fraction of the program that cannot be parallelised (the serial fraction).

$$\text{Speedup}(P) \leq 1 / (s + (1-s)/P)$$

Where:

- s = the serial fraction of the code (the portion that cannot be parallelised, between 0 and 1).
- P = the number of processors.
- (1-s) = the parallel fraction (the portion that CAN be parallelised).

10.2 Intuition and Implications

As P → ∞ (infinite processors), the parallel fraction takes zero time. The maximum speedup approaches 1/s. Even a tiny serial fraction becomes a hard ceiling:

Serial Fraction (s)	Maximum Possible Speedup (P → ∞)
Serial Fraction (s)	Maximum Possible Speedup (P → ∞)
50% (s = 0.5)	2x — even with infinite processors, the program can only run 2x faster.
10% (s = 0.1)	10x
5% (s = 0.05)	20x
1% (s = 0.01)	100x
0.1%(s = 0.001)	1000x

Important Concept: Amdahl's Law in Practice

Amdahl's Law explains why optimising the parallel portion of a program is not enough — the serial portions (initialisation, I/O, synchronisation overhead) become the bottleneck at scale. For example, if a program's startup and I/O takes 5% of the total time (s = 0.05), adding more than ~100 processors yields diminishing returns, and the maximum speedup is capped at 20x. This is why modern HPC work focuses on minimising serial fractions and reducing synchronisation overhead.

11. How an HPC Cluster Works End-to-End

A high-level understanding of HPC cluster operation is essential for any practitioner. Here is the typical workflow:

- 1. Cluster Configuration: The cluster consists of a Login Node (where users log in via SSH) and many Compute Nodes connected via a high-speed interconnect (e.g., 100 Gbps InfiniBand). Shared global storage (long-term and scratch) is accessible by all nodes.
- 2. Job Submission: The user submits a 'batch job' script to a job scheduler (e.g., SLURM). The script specifies resource requirements (nodes, cores, memory, time limit). The job is placed in a queue and dispatched to compute nodes when resources are available.
- 3. Task Parallelisation: The computational work is divided into independent tasks that run simultaneously across the allocated nodes.
- 4. Data Distribution: Input data is distributed across nodes. Each node works on its partition.
- 5. Parallel Computation: Nodes compute in parallel, communicating intermediate results via MPI calls over the interconnect when needed.
- 6. Monitoring and Control: The scheduler and monitoring tools track job progress, resource utilisation, and node health.
- 7. Output: The final result is collected and stored on the parallel file system, often rendered into visualisations for scientific analysis.

> **Key Insight**
> The Login Node is never used for computation — it is a lightweight access point only. Running compute jobs on the login node wastes shared resources and is considered poor practice. All heavy computation must go through the job scheduler.

12. Summary: Key Concepts and Exam Checklist

Topic	What You Should Know
Topic	What You Should Know
TOP500 & FLOP/s	Purpose of TOP500, Linpack benchmark, Rmax, the Exa/Peta/Tera/Giga scale.
Top Supercomputers	El Capitan (#1, APU), Frontier (#2, Exascale pioneer), Aurora (#3, Intel), JUPITER (#4, efficiency leader).
APU Architecture	Why fusing CPU+GPU on one die eliminates the PCIe bottleneck.
Flynn's Taxonomy	SISD (serial PC), SIMD (GPU/vector), MIMD (cluster). Modern HPC = hybrid SIMD + MIMD.
Shared vs Distributed Memory	SMP: easy but limited scalability. Clusters: scalable but requires explicit messaging.
OpenMP	Shared memory, fork-join model, #pragma directives, shared-by-default variables, data races.

MPI	Distributed memory, SPMD model, explicit MPI_Send/Recv, the standard for Exascale.
CUDA/HIP	GPU programming, offload compute kernels, PCIe vs NVLink bandwidth.
Concurrency vs Parallelism	Concurrency = interleaved (1 core); Parallelism = simultaneous (multiple cores).
5 Paradigms of Science	Theory, Experiment, Simulation, Data Analysis (HPDA), Machine Learning.
The 7 Dwarfs + 13 Motifs	Original 7 (DLA, SLA, FFT, N-Body, Structured Grid, Unstructured Grid, Monte Carlo) + 6 new ones.
Motif Bottlenecks	DLA = compute-bound; SLA = memory-bound; FFT = communication-bound; Monte Carlo = embarrassingly parallel.
Co-Design Principle	Motif analysis drives hardware design (GPUs for DLA, TPUs for AI, HBM for bandwidth).
Moore's Law	Transistor density doubles ~18 months. Since 2005, gains come from more cores, not faster clocks.
The Power Wall	Frequency stalled at ~3 GHz. Power $\propto f^3$. Multi-core is the only way forward.
Amdahl's Law	Speedup $\leq 1/(s + (1-s)/P)$. Serial fraction limits maximum speedup regardless of P.
HPC Cluster Workflow	Login node → job scheduler → compute nodes → interconnect → parallel file system.